

课程笔记

MIT 6.S191 Lecture 8: Large Language Models — Post-Training

基于公开课程资料整理

2025 年春季



视频作者/频道: Liquid AI (Guest Lecture)

发布日期: 2025-04-21

视频时长: 68 分钟

视频链接: https://www.youtube.com/watch?v=_HfdnCcbMOE

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：什么是 Post-Training	3
1.1 本章小结	3
2 Fine-Tuning 的类型与适用场景	4
2.1 三种 Fine-Tuning 类型	4
2.2 何时应该 Fine-Tune	4
2.3 本章小结	4
3 Post-Training 数据集：质量的三个维度	5
3.1 好数据的定义	5
3.2 多样性的重要性	5
3.3 复杂度提升：Auto-Evol	5
3.4 数据格式	5
3.5 本章小结	6
4 数据集生成流水线	7
4.1 Instruction Data 生成	7
4.1.1 Instruction Following 数据的特殊处理	7
4.2 Preference Data 生成：Ultra Feedback 范式	7
4.3 案例：Open Perfect Blend 数据集	7
4.4 Chat Template	8
4.5 本章小结	8
5 训练算法与技术	9
5.1 推荐的 Fine-Tuning 工具库	9
5.2 SFT 的三种训练技术	9
5.3 Preference Alignment 算法	9
5.3.1 PPO (Proximal Policy Optimization)	10
5.3.2 DPO (Direct Preference Optimization)	10
5.4 关键训练超参数	10
5.5 训练监控	11
5.6 本章小结	11
6 Model Merging：无需训练的模型增强	12
6.1 基本概念	12
6.2 主要合并技术	12
6.2.1 SLERP (Spherical Linear Interpolation)	12
6.2.2 DARE + TIES	12
6.3 Model Merging 的实际配置	12
6.4 应用案例：语言特定模型	13
6.5 本章小结	13

7	模型评估	14
7.1	评估的重要性	14
7.2	评估方法一: Automated Benchmarks	14
7.3	评估方法二: Human Evaluation (人类评估)	14
7.4	评估方法三: LLM-as-Judge	15
7.5	自建评估体系的建议	15
7.6	本章小结	15
8	Test-Time Compute Scaling	16
8.1	核心思想	16
8.2	Majority Voting (多数投票)	16
8.3	Best-of-N (最优选择)	16
8.4	Process Reward Model (PRM)	16
8.5	Scaling 实验结果	17
8.6	本章小结	17
9	总结与延伸	18
9.1	Post-Training 流程总览	18
9.2	核心要点回顾	18
9.3	方法选择的工程决策表	19
9.4	给实践者的落地顺序	19
9.5	拓展阅读	19

1 引言：什么是 Post-Training

本讲由 Liquid AI 的研究人员（《LLM Engineer's Handbook》作者）主讲，系统介绍了大语言模型（LLM）的 **Post-Training**（后训练）流程。Post-Training 是预训练之后的所有训练步骤，目标是将一个只会预测下一个 token 的 base model 转化为一个真正有用的 AI assistant。

在预训练阶段，模型在海量原始文本上以 next-token prediction 为目标进行训练，产生的 base model 虽然具备强大的语言建模能力，但并不具备回答问题或遵循指令的能力。Post-Training 正是弥补这一鸿沟的关键环节。

Post-Training 的两个核心阶段

1. **Supervised Fine-Tuning (SFT)**: 通过指令-回答对 (instruction-answer pairs) 教会模型遵循指令、以对话形式回答问题。模型在此阶段学习 chat template 结构 (system / user / assistant)。
2. **Preference Alignment (偏好对齐)**: 通过对比 chosen answer (期望回答) 与 rejected answer (不期望回答) 来调整模型行为，使其产出更符合人类偏好的回答。

Pre-Training vs. Post-Training 的数据规模差异

预训练使用数万亿 (trillions) tokens 的原始文本，数据集规模极为庞大。而 Post-Training 的数据集相对小得多，通常在数千到百万量级，但对数据**质量**的要求远高于数量。这一差异是理解 Post-Training 方法论的基础。

1.1 本章小结

Post-Training 是将 base model 转化为实用 assistant 的核心环节，包含 SFT 和 Preference Alignment 两个主要阶段。它的特点是数据规模远小于预训练，但对质量要求极高。

2 Fine-Tuning 的类型与适用场景

2.1 三种 Fine-Tuning 类型

讲者将 Fine-Tuning 分为三类，各自对应不同的数据规模需求和目标：

表 1: Fine-Tuning 类型对比

类型	目标	数据规模	示例
通用型 (General Purpose)	创建全能 chatbot	>100 万样本	ChatGPT, Claude
领域型 (Domain Specific)	嵌入领域知识	数千 ~ 数十万	医疗/法律/金融
任务型 (Task Specific)	学习单一功能	数百 ~ 数千	摘要、拼写检查

数据需求受多因素影响

上述样本数量仅为粗略估计。实际需求取决于：(1) **模型规模**——模型越大，所需样本越少；(2) **任务复杂度**——若 base model 对目标语言/领域的知识接近于零（如从未见过的小语种），fine-tuning 的效果会非常有限，因为缺乏足够 token 来”从头学习”。

2.2 何时应该 Fine-Tune

讲者建议遵循一个明确的决策流程：

1. 优先尝试 **In-Context Learning / RAG**：门槛最低，便于快速验证。
2. 若效果不满意，考虑以下四种 fine-tuning 动机：
 - **改变输出风格/格式**：如调整模型生成邮件的语气
 - **添加领域知识（浅层）**：如教模型关于你公司的事实
 - **知识蒸馏**：将 GPT-4 级别能力蒸馏到更小的模型中，降低成本和延迟
 - **提升特定任务质量**：如 diagram 生成，可在窄任务上超越 frontier model

企业选择 Fine-Tuning 的商业动机

除了技术层面，企业选择开源模型 + fine-tuning 路线还有两个重要的非技术原因：(1) **数据控制**——不希望将敏感数据通过 API 发送到云端；(2) **定制化**——希望深度定制模型的行为、语气和输出格式，创建专属模型。

2.3 本章小结

Fine-Tuning 分为通用型、领域型和任务型三个层次，数据需求依次递减。实际操作中应先尝试 in-context learning，确认不足后再考虑 fine-tuning。Fine-Tuning 的价值不仅在于技术提升，还在于给予企业对模型和数据控制权。

3 Post-Training 数据集：质量的三个维度

3.1 好数据的定义

讲者认为数据质量是整个 Post-Training 流程中最核心的问题，可以从三个维度来衡量：

数据质量的三维框架

1. **Accuracy (准确性)**：回答是否正确。对于简单问题较易验证；对于代码可通过 unit test 验证，对于数学可通过 solver 验证。
2. **Diversity (多样性)**：样本之间是否覆盖了用户可能的各种交互场景。多样性不足会导致模型在某些领域能力严重缺失。
3. **Complexity (复杂度)**：样本是否足够有挑战性，包含多步推理、chain-of-thought 等。

3.2 多样性的重要性

通过对不同数据集的 embedding 进行 2D 可视化，可以直观看到差异：

- **真实用户对话**（如 ShareGPT、WildChat）：embedding 分布覆盖极广，因为用户提问涉及方方面面
- **数学数据集**：embedding 高度聚集在单一区域
- **代码数据集**：同样集中在有限区域

因此，若目标是构建通用 LLM，仅有数学和代码数据远远不够，必须引入多样化的对话数据来扩展覆盖范围。

合成数据的多样性陷阱

合成数据（synthetic data）的一大风险在于多样性坍缩。少量合成数据可以有效扩充多样性，但**过多**合成数据反而会导致数据分布收窄、多样性急剧下降，最终降低整体数据质量。

3.3 复杂度提升：Auto-Evol

低复杂度样本如”埃菲尔铁塔多高？”只需几个词即可回答。可以使用 **Auto-Evol** 等框架，让 LLM 对 prompt 进行改写以提升复杂度——例如要求多角度分析、比较不同时期数据、解释工程原理等。改写后的问题需要更长、更详细的回答，自然提升了数据的复杂度。

3.4 数据格式

Post-Training 使用两种主要数据格式：

表 2: Post-Training 数据格式

格式	组成	用途
Instruction Data	system prompt + instruction + output	SFT
Preference Data	instruction + chosen answer + rejected answer	DPO/PPO

3.5 本章小结

数据质量由准确性、多样性和复杂度三个维度共同决定。在构建数据集时，需特别关注合成数据的多样性风险，并可使用 Auto-Evol 等工具提升样本复杂度。

4 数据集生成流水线

4.1 Instruction Data 生成

讲者介绍了一个典型的 instruction data 生成流水线：

1. **Seed Data**：起点可以是原始文本、已有的 QA 对、或用户真实查询
2. **指令生成**：若只有原始文本（即答案），可通过 **back translation**（反向翻译）让 LLM 生成对应的问题
3. **答案生成**：使用 LLM 基于指令生成回答
4. **评分与过滤**：
 - 使用启发式规则或 LLM-as-Judge 评估质量
 - 数据去重（deduplication）
 - 去污染（decontamination）—— 确保不包含测试集样本
 - 关键词过滤—— 移除不需要的内容

4.1.1 Instruction Following 数据的特殊处理

Instruction following 是一类专注于**约束遵循**的数据。例如：“用全小写英文回答”。生成此类数据时，可以将约束条件附加到 prompt 之后，让 LLM 生成回答，然后用**程序化测试**验证约束是否被满足（如语言检测、大小写检查），筛除不符合要求的样本。

4.2 Preference Data 生成：Ultra Feedback 范式

Preference data 的生成采用不同策略：

1. 给定 prompt，查询**多个不同 LLM**（而非单一模型）
2. 使用 Judge LLM 对每个回答评分
3. 取最高分的回答作为 **chosen**，最低分的作为 **rejected**
4. 去重、移除过短回答等后处理

Decontamination（去污染）

去污染是数据生成流水线中至关重要的一步。例如，IFEval 是一个广泛使用的 instruction following 评测集。在生成训练数据时，必须确保生成的样本与测试集不存在高相似度，否则会导致评测分数虚高但实际能力并未提升。

4.3 案例：Open Perfect Blend 数据集

讲者展示了自己制作的 Open Perfect Blend 数据集，这是一个面向通用 fine-tuning 的混合数据集。其类别配比大致为：

- **Math**（数学）：较大比例

- **Code** (代码): 较大比例
- **Chat** (通用对话): 中等比例
- **Instruction Following**: 较小比例

这种混合比例 (data mixture) 的设计直接决定了模型在不同能力维度上的表现。

4.4 Chat Template

数据准备完成后的最后一步是应用 **Chat Template**。这是让模型理解“谁在说话”的关键结构：

```
1 <|im_start|>system
2 You are a helpful assistant.
3 <|im_end|>
4 <|im_start|>user
5 What is the capital of France?
6 <|im_end|>
7 <|im_start|>assistant
8 The capital of France is Paris.
9 <|im_end|>
```

Listing 1: Chat Template 示例 (ChatML 格式)

特殊 token `<|im_start|>` 和 `<|im_end|>` 标记每条消息的起止，角色标签 (system / user / assistant) 明确说话者身份。多轮对话通过重复 user-assistant 对实现。

Chat Template 是 SFT 的核心学习目标

Chat Template 的结构是 SFT 阶段模型需要学习的核心内容之一。正是这种结构化的对话格式让 base model 从“只会续写文本”进化为“能进行多轮对话的 assistant”。不同模型家族 (Llama, Mistral, Qwen 等) 使用不同的 chat template，但原理相同。

4.5 本章小结

数据集生成是一个系统化的流水线，包括种子数据准备、指令/答案生成、评分过滤、去重去污染等步骤。Preference data 通过多模型采样 + Judge 评分的方式获取。最终需将数据格式化为 Chat Template 供 SFT 训练使用。

5 训练算法与技术

5.1 推荐的 Fine-Tuning 工具库

讲者推荐了三个主流的 fine-tuning 工具库：

表 3: 主流 Fine-Tuning 工具库对比

工具	开发方	特点
TRL	Hugging Face	基于 Transformers, 算法种类最全, 与最新研究保持同步
Axolotl	社区	基于 TRL, YAML 配置驱动, 易于分享和复现训练配置
Unsloth	社区	单 GPU 优化, 极高效率, 内置量化等实用功能

5.2 SFT 的三种训练技术

Full Fine-Tuning vs. LoRA vs. QLoRA

- **Full Fine-Tuning**: 全精度加载模型, 训练 100% 参数。质量最高但 VRAM 需求极大, 通常需要整个 GPU 集群。
- **LoRA (Low-Rank Adaptation)**: 冻结原始模型权重, 仅训练附加的低秩适配器矩阵 A 和 B 。仅需训练约 0.5% 的参数, 速度快、VRAM 需求低, **推荐作为默认选择**。
- **QLoRA (Quantized LoRA)**: 先将模型量化为 4-bit 精度再加载, 然后应用 LoRA。进一步降低 VRAM 需求, 但训练速度更慢, 性能有轻微下降 (几个百分点)。

LoRA 的数学原理: 对于原始权重矩阵 $W \in \mathbb{R}^{d \times k}$, LoRA 添加低秩分解:

$$W' = W + \Delta W = W + BA$$

其中:

- $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$
- $r \ll \min(d, k)$ 为秩 (rank), 通常取 8-64
- 训练时仅更新 A 和 B , 原始 W 被冻结

Full Fine-Tuning 通常是过度的

除非你是一家专门训练 LLM 的公司, 否则 LoRA 几乎总是更好的选择。Full Fine-Tuning 可能导致过拟合, 且性能提升往往不值得额外的计算开销。如果 LoRA 的 VRAM 需求仍然过高, 才考虑退而使用 QLoRA。

5.3 Preference Alignment 算法

偏好对齐领域有超过 100 种不同算法, 但讲者建议只需记住两个:

5.3.1 PPO (Proximal Policy Optimization)

PPO 是经典的 RLHF 算法，需要三个模型同时加载：

1. **Frozen Model** (参考策略)：训练前的模型快照，用于计算 KL divergence
2. **Train Model** (当前策略)：接收 prompt 生成回答的模型
3. **Reward Model**：对生成的回答进行评分

训练流程：Train Model 生成回答 → Reward Model 评分 → 使用 KL divergence 约束策略更新幅度 → 更新 Train Model 参数。

优点是质量上限高，缺点是需要三个模型、极其昂贵且调参困难。

5.3.2 DPO (Direct Preference Optimization)

DPO 只需要两个模型（若使用 LoRA 则实际上只需加载一个基础模型 + 两组适配器）：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

其中：

- y_w 为 chosen answer, y_l 为 rejected answer
- π_{θ} 为训练中的策略, π_{ref} 为参考策略
- β 控制偏离参考策略的惩罚强度

实践建议：DPO 优于 PPO

除非拥有 OpenAI 级别的无限资源，否则推荐使用 DPO。DPO 更快、更便宜、更易调参，质量仅略低于 PPO。DPO 不需要单独的 reward model，直接在 preference data 上优化，大幅简化了训练流程。

5.4 关键训练超参数

表 4: Post-Training 关键超参数

参数	推荐值	说明
Learning Rate	$1 \times 10^{-5} \sim 5 \times 10^{-5}$	最关键参数，过高导致 loss spike
Batch Size	受 VRAM 限制	可用 gradient accumulation 模拟大 batch
Max Length	受 VRAM 限制	决定输入序列的最大长度
Epochs	3~5	完整遍历训练集的次数
Optimizer	AdamW	强基线，通常无需更换
Attention	Flash Attention	高效注意力实现，尤其加速长序列处理

Gradient Accumulation 与 Effective Batch Size

当 GPU 内存不足以支持期望的 batch size 时，可以使用 gradient accumulation：在多个 forward pass 后累积梯度，再统一更新权重。这样 **effective batch size** = per-GPU batch size × accumulation steps × GPU 数量，能在不增加内存的前提下模拟更大的 batch。

5.5 训练监控

训练过程中需要监控三个核心指标：

1. **Training Loss**：应平滑下降，若出现 loss spike 通常意味着 learning rate 过高
2. **Validation Loss**：用 1-2% 的数据作为验证集，监控是否过拟合
3. **Gradient Norm**：梯度的幅值，过多 spike 可能暗示数据问题

Loss Spike 诊断

如果训练初期 loss 平稳下降，但随后突然出现大幅跳升 (spike)，最可能的原因是 **learning rate 过高**。解决方法是降低 learning rate，应能获得更平滑的 loss 曲线。注意不要将训练初期的微小波动误判为 spike——小幅初始波动通常是正常现象。

5.6 本章小结

SFT 推荐使用 LoRA 作为默认训练技术，Preference Alignment 推荐使用 DPO。关键超参数中 learning rate 最为重要，AdamW + Flash Attention 是可靠的默认选择。训练过程中应密切监控 training loss、validation loss 和 gradient norm。

6 Model Merging: 无需训练的模型增强

6.1 基本概念

Model Merging 是一种将多个模型的参数直接组合的技术，**无需额外训练**。最初被视为“玩笑”，但现在已被每家 LLM 公司采用。

核心思想：取多个经过不同 fine-tuning 的同架构模型，通过数学操作（如加权平均）合并其参数，得到一个综合多方优势的新模型。

Model Merging 的限制条件

- 只能合并**相同架构、相同规模**的模型（如只能 Llama 3 合并 Llama 3，不能跨代合并）
- 建议使用**全精度**模型进行合并——量化模型的精度损失会传播到合并结果
- 选择合适的权重和参数更像是“炼金术”而非严格科学——需要反复实验

6.2 主要合并技术

6.2.1 SLERP (Spherical Linear Interpolation)

SLERP 在球面上进行插值（而非直线插值），通常比简单的线性平均产生更好的合并效果。

$$\text{SLERP}(\mathbf{p}_0, \mathbf{p}_1; t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{p}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{p}_1$$

其中 $\Omega = \arccos(\mathbf{p}_0 \cdot \mathbf{p}_1 / \|\mathbf{p}_0\| \|\mathbf{p}_1\|)$ ， $t \in [0, 1]$ 为插值系数。

特点：简单可靠，在开源社区非常流行；但**只能合并两个模型**。

6.2.2 DARE + TIES

更高级的合并技术，可处理两个以上的模型：

- **DARE (Drop And REscale)**：随机剪枝并重新缩放源模型的参数
- **TIES (TrIm, Elect Sign, merge)**：保留最显著的参数，并通过 **sign consensus**（符号一致性）解决参数正负相消的问题

Sign consensus 的动机：若模型 A 的某参数为正、模型 B 的相同参数为负，简单平均得到接近零的结果，实际上丢失了两个模型各自学到的信息。TIES 通过投票机制确定参数符号，避免这种抵消。

6.3 Model Merging 的实际配置

使用 **mergekit** 等工具，通过 YAML 配置文件定义合并方案：

- **density**：控制保留参数的比例（TIES 方法中不保留全部参数）
- **weight**：控制各源模型在合并中的权重（可根据评估分数赋予不同权重）

讲者展示了 Daredevil 8B 模型的家族树——通过迭代合并（合并的模型再与其他模型合并）可以构建出复杂的模型谱系，最终产物在多个 benchmark 上超越所有源模型。

6.4 应用案例：语言特定模型

以构建芬兰语 LLM 为例的完整流程：

1. 在芬兰语原始文本上做 continual pre-training (5-100B tokens)
2. 在芬兰语 instruction data 上做 SFT
3. 在芬兰语 preference data 上做 Preference Alignment
4. 问题：模型芬兰语能力大增，但**其他所有能力严重退化**
5. 解决方案：将芬兰语模型与通用 instruct 模型（如 Llama 3 Instruct）合并
6. 结果：获得一个**既擅长芬兰语、又保持通用能力**的模型

Model Merging 的核心价值

Model Merging 能够将不同模型的专长**叠加**在一起，而不会牺牲各自的已有能力。这是解决 fine-tuning 过程中”灾难性遗忘”（catastrophic forgetting）问题的一种高效策略，尤其适合语言特定模型、领域特定模型等场景。

6.5 本章小结

Model Merging 是一种零训练成本的模型增强技术，通过参数级别的数学操作组合多个模型的优势。SLERP 适合两模型合并，DARE+TIES 支持多模型合并。它是解决 fine-tuning 导致能力退化问题的有力工具。

7 模型评估

7.1 评估的重要性

讲者坦言 LLM 评估是整个领域最大的难题之一——“我们其实不太知道自己在做什么”。但评估依然至关重要，因为 Post-Training 的本质是**优化**，如果评估指标不对，优化方向就会偏离。

7.2 评估方法一：Automated Benchmarks

原理：给定问题集（如 MMLU 的多选题），模型选择答案，以准确率等指标计分。

表 5: 常见自动化 Benchmark

Benchmark	领域	指标
MMLU	通用知识（57 个学科）	准确率
GSM8K	小学数学推理	准确率
MATH	竞赛数学	准确率
HumanEval	代码生成	pass@k
IFEval	Instruction Following	约束满足率

优点：可扩展、低成本、可复现、可针对特定任务设计。

缺点：不反映模型的实际使用方式——用户与模型是**对话**，而非做选择题。

7.3 评估方法二：Human Evaluation（人类评估）

原理：人类评审员在两个匿名模型的回答中选择更好的那个（如 Chatbot Arena）。

优点：

- 直接反映人类偏好，是 Post-Training 的终极优化目标
- 可精细控制评估维度（如只关注毒性、只关注准确性）
- 数据污染风险低

缺点：

- 极难规模化，成本高、耗时长
- 人类评审有**严重偏见**：更长的回答、更自信的语气会被偏好，即使内容有误

人类偏好与自动化 Benchmark 不相关

讲者展示了一项关键发现：Chatbot Arena 排名与 MMLU、GSM8K、HumanEval 等自动化 benchmark 的相关性非常低。一个 MMLU 高分的模型可能在人类偏好上表现很差，反之亦然。因此，**必须同时使用两类评估方法**才能全面了解模型能力。

7.4 评估方法三：LLM-as-Judge

为了在规模化和人类偏好之间找到折中，可以使用 LLM 替代人类进行评估：

- **单 Judge**：一个强力 LLM（如 GPT-4）对回答评分
- **多 Judge (Jury)**：多个 LLM 组成评审团，投票决定优劣——更稳健可靠

优点：与人类偏好高度相关，能处理自由形式的复杂任务，提供直接反馈。

缺点：LLM judge 自身也有偏见（且与人类偏见高度相似），仍需少量人类评估来校验 judge 的一致性。

7.5 自建评估体系的建议

1. **尽早开始**——在 fine-tuning 之前就设计评估，类似 test-driven development
2. **持续迭代**——评估数据集不是一成不变的，需根据模型表现不断修正
3. **混合使用**——自动化 benchmark + LLM-as-Judge + 少量人类评估
4. **横向比较**——不仅比较自己的各版本 fine-tune，也与其他模型和架构对比

评估驱动的迭代开发

讲者强调 Post-Training 的时间分配应大致为：数据准备占三分之一，训练占三分之一，评估占三分之一。评估不是事后验证，而是贯穿整个流程的核心环节。每轮评估的结果直接指导下一轮数据和训练的改进方向。

7.6 本章小结

LLM 评估主要有三种方法：自动化 benchmark（可扩展但不反映真实使用）、人类评估（最接近目标但难以规模化）、LLM-as-Judge（折中方案）。三者互补，应组合使用。人类偏好与自动化 benchmark 的低相关性是设计评估体系时必须认识到的关键事实。

8 Test-Time Compute Scaling

8.1 核心思想

Training-time scaling（增加训练数据和算力）已被证明有效。Test-Time Compute Scaling 将同样的理念应用于推理阶段：**在推理时投入更多计算来换取更高质量的输出。**

8.2 Majority Voting（多数投票）

最简单的 test-time scaling 方法：

1. 对同一个问题（如数学题），让模型生成 N 个不同回答（使用非零温度以获得多样性）
2. 取出现频率最高的答案作为最终输出

直觉：由于 top-p 采样引入的随机性，模型偶尔会选择错误的 token 导致整条推理链偏离。通过多次采样，正确答案大概率会更频繁地出现。

8.3 Best-of-N（最优选择）

Majority Voting 的改进版：

1. 生成 N 个不同回答
2. 使用 **Reward Model** 或 **Judge LLM** 对每个回答评分
3. 选择得分最高的回答

相比 Majority Voting，Best-of-N 利用了外部评估信号，通常效果更好，但需要额外运行一个评估模型。

8.4 Process Reward Model (PRM)

更精细的方法——不评估整体答案，而是评估**每个推理步骤**：

Process Reward Model 的工作流程

1. LLM 生成部分解答（partial solution）
2. PRM 对每个步骤评分——“分数代表”该步骤最终导向正确答案的概率”
3. 选择得分最高的步骤继续扩展
4. 重复上述过程，逐步构建完整解答

这形成了一种**树搜索**（tree search）机制：在推理步骤的空间中进行搜索，由 PRM 引导搜索方向。

讲者用一个 base-8 乘法的例子演示了这一过程：

- Attempt 1：第一步正确，但第二步在十进制而非八进制下计算，PRM 拒绝
- Attempt 2：多了一步正确步骤，但最终仍犯同样错误，PRM 再次拒绝
- Attempt 4：所有步骤均正确，PRM 通过

8.5 Scaling 实验结果

Hugging Face 团队的实验显示：

- 使用 Llama 3.2 1B 和 3B（小模型），通过增加每题的生成次数
- 在数学 benchmark 上，小模型 + test-time scaling 可以**超越** Llama 3.1 8B 甚至 Llama 3.1 70B

推理速度与输出质量的权衡

Test-Time Compute Scaling 的本质是用**推理时间**（inference speed）换**输出质量**（output quality）。这意味着一个小而高效的模型 + 充足的推理预算，可以在特定任务上匹敌甚至超越体量大得多的模型。这对资源受限的场景具有重要意义。

Reward Model 从何而来

Test-Time Compute Scaling 中使用的 Reward Model 可以复用 Preference Alignment 阶段（PPO 训练流程中）训练的 reward model。这个模型的输入是文本，输出是 0 到 1 之间的质量分数。也可以直接用 Judge LLM（通过 prompting）替代专门训练的 reward model。

8.6 本章小结

Test-Time Compute Scaling 是 2025 年初最重要的趋势之一，核心是在推理时投入更多计算以提升输出质量。从简单的 Majority Voting 到 Best-of-N，再到基于 Process Reward Model 的步骤级搜索，方法复杂度和效果依次递增。实验证明小模型 + test-time scaling 可超越大模型。

9 总结与延伸

9.1 Post-Training 流程总览

整个 Post-Training 是一个迭代循环，由三个等重的环节构成：

1. **数据构建** (约 1/3 时间)：设计数据混合比例，确保准确性、多样性和复杂度，通过 seed data + LLM 生成 + 过滤的流水线批量生产高质量训练数据
2. **模型训练** (约 1/3 时间)：SFT (推荐 LoRA) → Preference Alignment (推荐 DPO) → 可选的 Model Merging
3. **评估迭代** (约 1/3 时间)：自动化 benchmark + LLM-as-Judge + 人类评估三管齐下，评估结果反馈到下一轮数据和训练的改进

9.2 核心要点回顾

本讲核心 Takeaways

- Post-Training 将 base model 转化为有用的 assistant，核心步骤为 SFT + Preference Alignment
- 数据质量 > 数据数量，三个维度：Accuracy, Diversity, Complexity
- LoRA 是大多数场景下的最佳 SFT 技术，DPO 是最实用的对齐算法
- Model Merging 是零成本组合多模型优势的利器
- 人类偏好与 automated benchmark 相关性低，必须组合使用多种评估方法
- Test-Time Compute Scaling 用推理时间换输出质量，是当前最重要的趋势

9.3 方法选择的工程决策表

目标场景	优先方法	关键判断标准
先快速验证需求	In-context learning / RAG	不要过早进入 fine-tuning, 先看 base model 是否已足够好
需要稳定输出格式或语气	SFT (通常配 LoRA)	样本质量是否足够高, 输出风格是否可被明确示范
需要让回答更符合人类偏好	DPO / Preference Alignment	是否有 chosen vs rejected 的偏好数据, 评估是否覆盖真实用户偏好
需要组合多个模型专长	Model Merging	各模型能力是否互补, 以及合并后是否有退化风险
推理阶段还有预算	Test-Time Compute Scaling	延迟和成本是否允许用更多采样换更高质量

表 6: Post-Training 主要方法的选择顺序

9.4 给实践者的落地顺序

如果把这一讲转成工程上的执行建议, 最稳妥的顺序通常是:

1. 先用最小成本的 prompting / RAG 确认问题边界
2. 再用高质量小数据集做 SFT, 解决风格和格式问题
3. 若要进一步提升可用性, 再引入偏好数据做 DPO
4. 最后根据成本、延迟与效果, 选择 model merging 或 test-time scaling 作为增强手段

9.5 拓展阅读

- **LLM Engineer's Handbook**——讲者撰写的系统性参考书
- **Open Perfect Blend** (Hugging Face)——通用 SFT 数据集混合方案
- **mergekit**——开源模型合并工具
- **TRL** (Hugging Face)——最全面的 Post-Training 工具库
- Rafailov et al., *Direct Preference Optimization* (NeurIPS 2023)——DPO 原始论文
- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models* (ICLR 2022)——LoRA 原始论文
- Snell et al., *Scaling LLM Test-Time Compute* (2024)——Test-Time Compute 的理论基础
- Chatbot Arena (<https://arena.lmsys.org>)——基于人类偏好的模型排名